

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | eric.maniraguha@auca.ac.rw | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

April 2025

Database development with PL/SQL

Reference reading



[11 Tuning PL/SQL Applications for Performance](#)

[PL SQL Performance Tuning](#)

[12 PL/SQL Optimization and Tuning](#)

[difference of explain plan, autotrace and tkprof](#)

[Youtube Tutorial: tkprof for trace analysis](#)

Lecture 11 – PL/SQL Performance Tuning

PL/SQL Performance Tuning: Key Concepts and Strategies



Performance tuning in PL/SQL is the systematic process of analyzing and optimizing PL/SQL code and associated SQL queries to **reduce execution time**, **minimize resource usage**, and **improve overall responsiveness** of database applications.

Objectives of PL/SQL Performance Tuning

1. Write Efficient PL/SQL Code

- Use best practices to enhance execution speed.
- Refactor existing code to remove inefficiencies.
- Avoid unnecessary computations and round-trips to the database.

2. Understand PL/SQL Optimization

- Learn how the PL/SQL compiler performs optimization.
- Configure and utilize different **optimization levels**.
- Know when and how to adjust compiler parameters for best performance.

3. Avoid Common Performance Issues

- Detect and fix:
 - ✓ Inefficient SQL statements.
 - ✓ Poorly written joins and subqueries.
 - ✓ Overuse of cursors or unnecessary procedural logic.
- Use **indexes**, **gathered statistics**, and **bind variables** effectively.



4. Optimize Loop Performance

- Replace row-by-row processing with:
 - ✓ FORALL for efficient DML operations.
 - ✓ BULK COLLECT for fast data retrieval.
- Minimize loop iterations and context switches.

5. Improve Function Calls

- Use **NOCOPY** to pass parameters by reference.
- Leverage **result caching** for frequently called functions.
- Apply **function-based indexes** to optimize query execution.

6. Leverage Built-In Tools and Features

- Analyze execution plans with EXPLAIN PLAN.
- Use TKPROF, SQL Trace, and DBMS_PROFILER to detect bottlenecks.
- Utilize DBMS_SQLTUNE and ADDM for automatic tuning insights.

7. Minimize Overheads

- Eliminate:
 - ✓ Implicit datatype conversions.
 - ✓ Redundant or expensive string operations.
 - ✓ Repetitive conditional logic or unnecessary checks.

8. Benchmark and Trace Performance

- Profile applications using:
 - ✓ DBMS_HPROF, DBMS_PROFILER, or external profilers.
- Track runtime behavior and memory usage.
- Perform baseline comparisons to measure improvement.

9. Compile for Native Execution

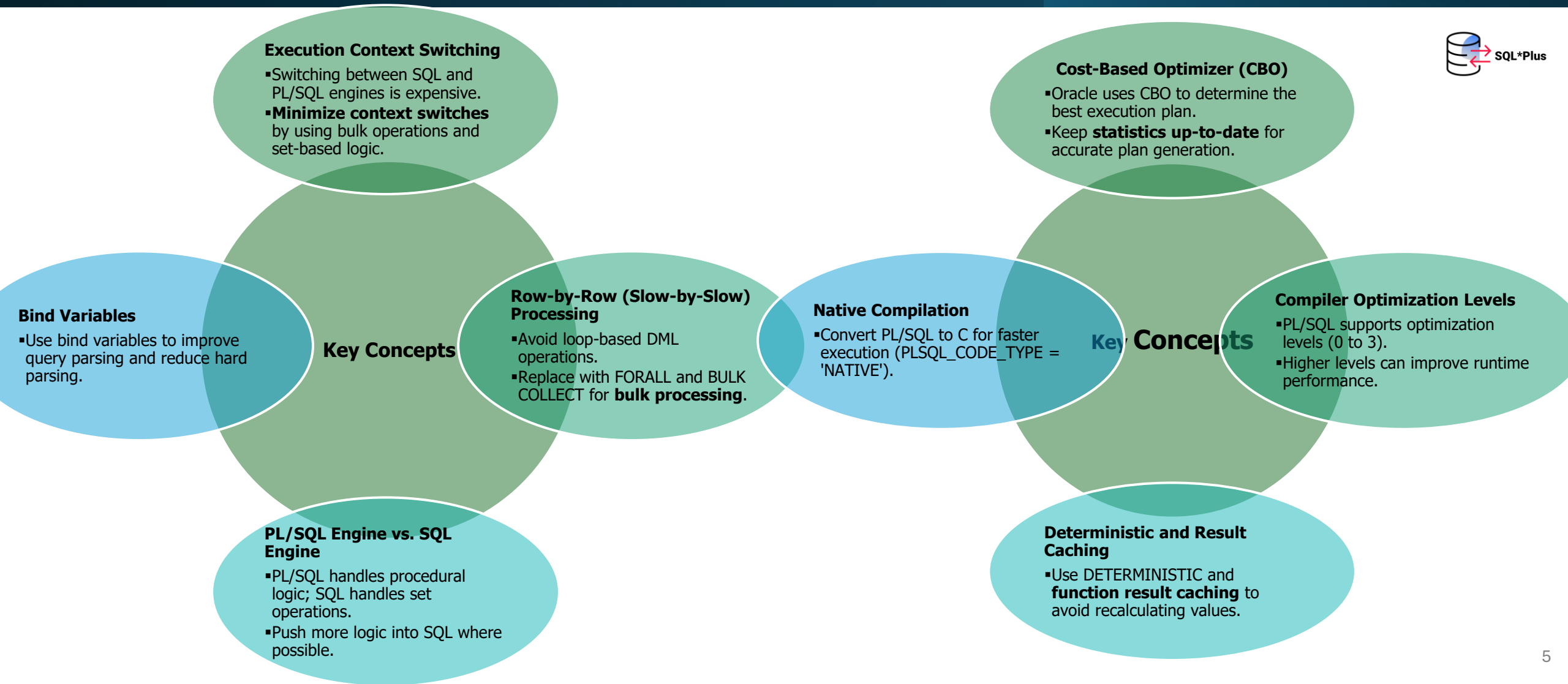
- Enable **PL/SQL Native Compilation** to translate PL/SQL to native code.
- Improve CPU-bound program execution.

10. Utilize Advanced Features

- Use **dynamic SQL** and EXECUTE IMMEDIATE smartly.
- Implement **table functions** and **pipelined functions** for set-based processing.
- Take advantage of **autonomous transactions** and **PRAGMA directives** when appropriate.



PL/SQL Performance Tuning: Key Concepts



PL/SQL Performance Tuning: Strategies



--Bulk Operations

```
FORALL i IN 1..l_data.COUNT  
INSERT INTO target_table VALUES l_data(i);
```

1. Use Bulk Operations

```
FORALL i IN 1..l_data.COUNT INSERT INTO target_table VALUES  
l_data(i);
```

- Reduces context switches and improves DML performance.

2. Optimize Loops

- Avoid unnecessary loops.
- Use collection operations like BULK COLLECT, FORALL, and LIMIT clause to fetch in batches.

3. Avoid Unnecessary SQL Calls

- Cache frequently accessed values.
- Use PL/SQL tables or associative arrays.

4. Minimize Use of Cursors When Possible

- Implicit cursors are often faster than explicit ones for simple queries.

5. Use Efficient SQL

- Write optimized SQL with appropriate indexes, joins, and WHERE clauses.
- Avoid SELECT * and unnecessary subqueries.

Performance
Tuning Strategies

Performance
Tuning Strategies

6. Optimize Functions and Procedures

- Use NOCOPY to reduce parameter-passing overhead.
- Mark functions DETERMINISTIC when applicable.
- Avoid calling PL/SQL functions in SQL if possible—inline logic instead.

7. Trace and Analyze Performance

- Tools:
 - ✓ EXPLAIN PLAN – Understand query execution paths.
 - ✓ TKPROF – Format SQL trace output.
 - ✓ DBMS_PROFILER, DBMS_HPROF – Profile PL/SQL code execution.

8. Manage Memory Wisely

- Limit collection sizes.
- Reuse collections and avoid memory leaks.

9. Use Result Caching

```
FUNCTION get_tax_rate(p_region VARCHAR2) RETURN NUMBER  
RESULT_CACHE RELIES_ON(tax_table)
```

- Cache frequently used function results.

10. Keep Statistics Updated

- Run DBMS_STATS.GATHER_TABLE_STATS regularly to help the optimizer choose the best execution plan.

--Use Result Caching

```
FUNCTION get_tax_rate(p_region VARCHAR2) RETURN  
NUMBER  
RESULT_CACHE RELIES_ON(tax_table)
```

PL/SQL Performance Tuning Concepts Illustrated

Tuning Concept	Description	Example
Optimize SQL Queries	Use only necessary columns, avoid <code>SELECT *</code> to reduce I/O and parsing time.	▪ <code>SELECT * FROM employees WHERE department_id = 10;</code> ▪ <code>SELECT employee_id, first_name, last_name FROM employees WHERE department_id = 10;</code>
Indexing	Create indexes on columns used in <code>WHERE</code> , <code>JOIN</code> , or <code>ORDER BY</code> to enhance retrieval speed.	<code>CREATE INDEX idx_department_id ON employees(department_id);</code> Then: <code>SELECT employee_id, first_name FROM employees WHERE department_id = 10;</code>
NOCOPY Hint	Reduces memory copying for large <code>IN OUT</code> or <code>OUT</code> parameters. Improves speed, especially in loops.	<code>PROCEDURE process_large_data(p_data IN OUT NOCOPY my_table_type) IS BEGIN -- Process END;</code>
Transaction Management	Avoid long-running transactions. Commit or roll back promptly to free locks and maintain integrity.	<code>plsql BEGIN UPDATE employees SET salary = salary * 1.10 WHERE department_id = 10; COMMIT; EXCEPTION WHEN OTHERS THEN ROLLBACK; END;</code>
Optimize Function Calls	Avoid repetitive expensive function calls in SQL; reduce calls by filtering or restructuring logic.	▪ <code>SELECT DISTINCT(SQRT(department_id)) FROM employees;</code> ▪ <code>SELECT SQRT(department_id) FROM (SELECT DISTINCT department_id FROM employees);</code>
Function-Based Index	Use when functions are used in <code>WHERE</code> clauses. Caches computed values for faster retrieval.	<code>CREATE INDEX idx_sqrt_dept_id ON employees(SQRT(department_id));</code>
Use NOCOPY for Parameters	Apply <code>NOCOPY</code> on large <code>OUT/IN OUT</code> parameters to avoid performance costs from copying on exceptions.	<code>PROCEDURE process_data(p_data IN OUT NOCOPY my_table_type);</code>
Self NOCOPY for Objects	When modifying object attributes, use <code>SELF IN OUT NOCOPY</code> to prevent redundant copying.	<code>plsql TYPE employee_obj_type IS OBJECT (id NUMBER, name VARCHAR2(100)); MEMBER PROCEDURE update_name(SELF IN OUT NOCOPY employee_obj_type, new_name VARCHAR2);</code>

PL/SQL code for Optimizing



In previous lectures, we explored several PL/SQL optimization techniques:

- We covered **loops and bulk processing**, focusing on how to efficiently handle large datasets using:
 - ✓ **Cursors**
 - ✓ **BULK COLLECT**
 - ✓ **FORALL**
 - ✓ **Compound Triggers**

These methods help reduce context switches between SQL and PL/SQL and significantly improve performance when processing large volumes of data.

In **Lecture 10**, we introduced:

- **Pipelined table functions**, which allow processing and returning rows one at a time, enabling efficient memory usage and streaming behavior.
- **Object types**, which support structured, modular, and reusable data representations in PL/SQL programs.

These techniques are essential for writing **high-performance, scalable, and maintainable PL/SQL applications**.



TKPROF and EXPLAIN PLAN in PL/SQL Performance Tuning



TKPROF Tool (Trace Kernel Profiler)



 **TKPROF** is a diagnostic tool that converts raw trace files (from SQL Trace) into readable reports, helping you analyze the performance of SQL statements.

Key Features of TKPROF



 **Execution Plan:** Displays how Oracle executes SQL statements.

 **Statistics Storage:** Creates a script to store performance stats in a database table for further analysis.

 **Detailed Analysis:** Highlights SQL statements consuming the most resources.

Benefits of Using TKPROF



 Detects **inefficient queries** by examining CPU, I/O, and elapsed time.

 Helps **combine multiple trace files** for a complete performance view.

 Makes it easier to **pinpoint bottlenecks** in PL/SQL and SQL code.

EXPLAIN PLAN in PL/SQL Performance Tuning

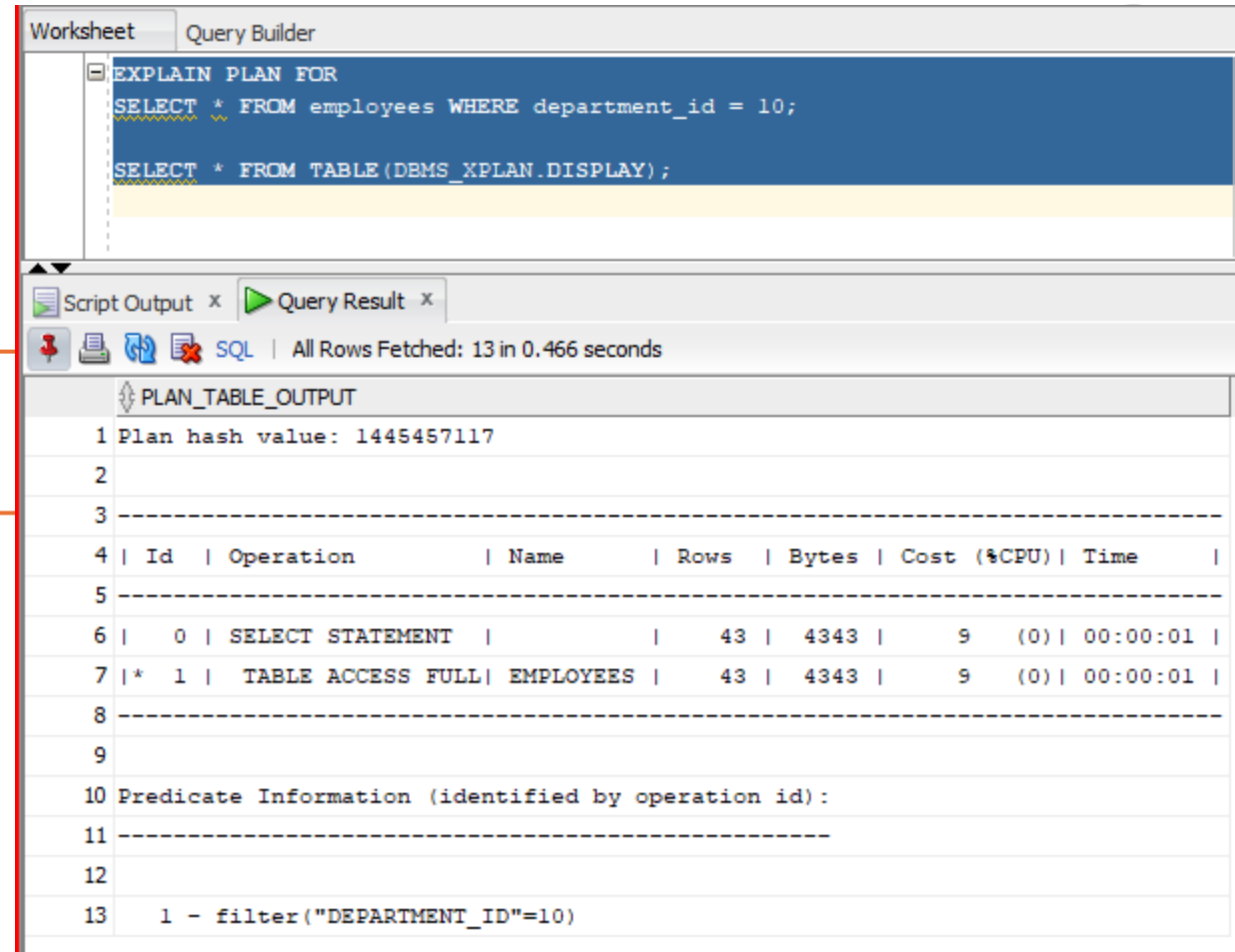
EXPLAIN PLAN

- A SQL statement that **shows the execution path** Oracle will take to run a query.
- Helps you understand the optimizer's decisions: whether it uses indexes, joins, full table scans, etc.

```
EXPLAIN PLAN FOR  
SELECT * FROM employees WHERE department_id = 10;  
  
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

When to Use

- Before running a query, use **EXPLAIN PLAN** to anticipate performance.
- After running a query, use **TKPROF** to see **actual execution stats** and performance costs.



The screenshot shows the Oracle SQL Developer interface. The 'Query Builder' tab is active, displaying the following SQL script:

```
EXPLAIN PLAN FOR  
SELECT * FROM employees WHERE department_id = 10;  
  
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Below the script, the 'Script Output' and 'Query Result' tabs are visible. The 'Query Result' tab shows the output of the EXPLAIN PLAN query, which is a table named 'PLAN_TABLE_OUTPUT'. The output is as follows:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		43	4343	9 (0)	00:00:01
* 1	TABLE ACCESS FULL	EMPLOYEES	43	4343	9 (0)	00:00:01

Below the table, the 'Predicate Information (identified by operation id):' is shown, with the following entry:

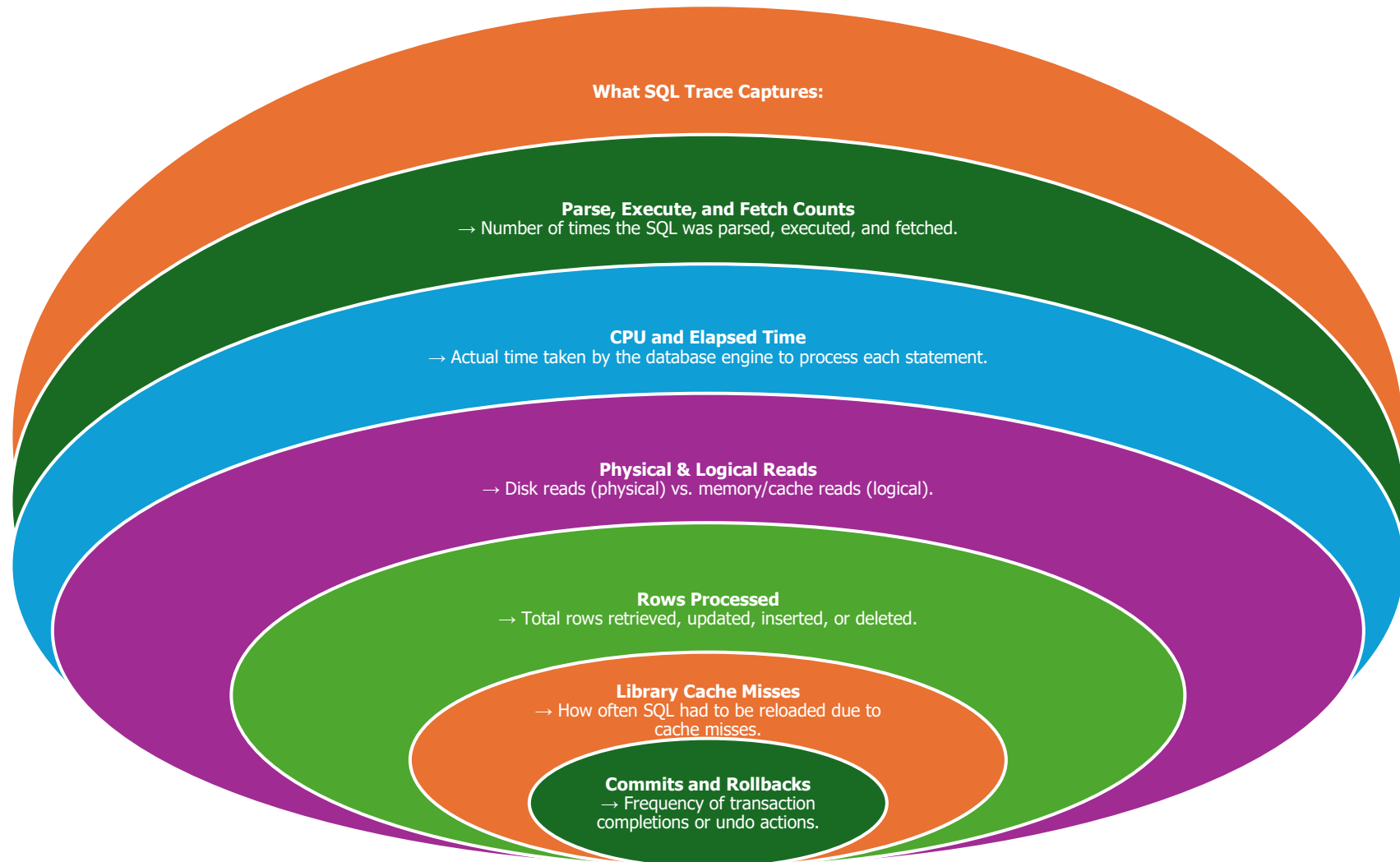
```
1 - filter("DEPARTMENT_ID"=10)
```

SQL Performance Analysis: Tracing and TKPROF Explained

When tuning SQL performance in Oracle, both **SQL Trace** (used with **TKPROF**) and **EXPLAIN PLAN** are valuable tools—but they serve different purposes and measure different performance aspects.

SQL Trace + TKPROF: Runtime Performance Analysis

The **SQL Trace** facility captures **actual execution details** for SQL statements, and **TKPROF** transforms this raw trace data into a readable format for analysis.



TKPROF Tool Interpreting Trace Data & Summary

TKPROF converts raw trace files into **human-readable reports** with detailed statistics and execution plans.



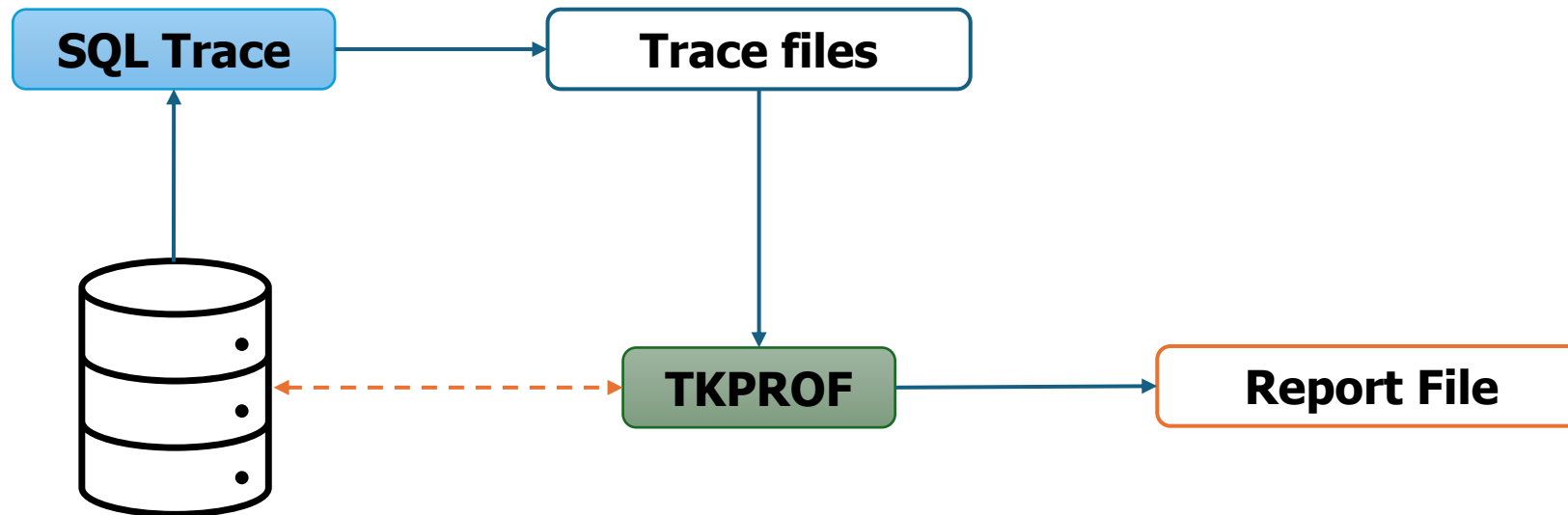
Key Features of TKPROF:

- Shows **actual runtime behavior**, not just optimizer predictions.
- Provides detailed **I/O and CPU usage** per SQL statement.
- Identifies **resource-heavy statements** for optimization.
- Reports how frequently each SQL statement was run and how much it consumed.

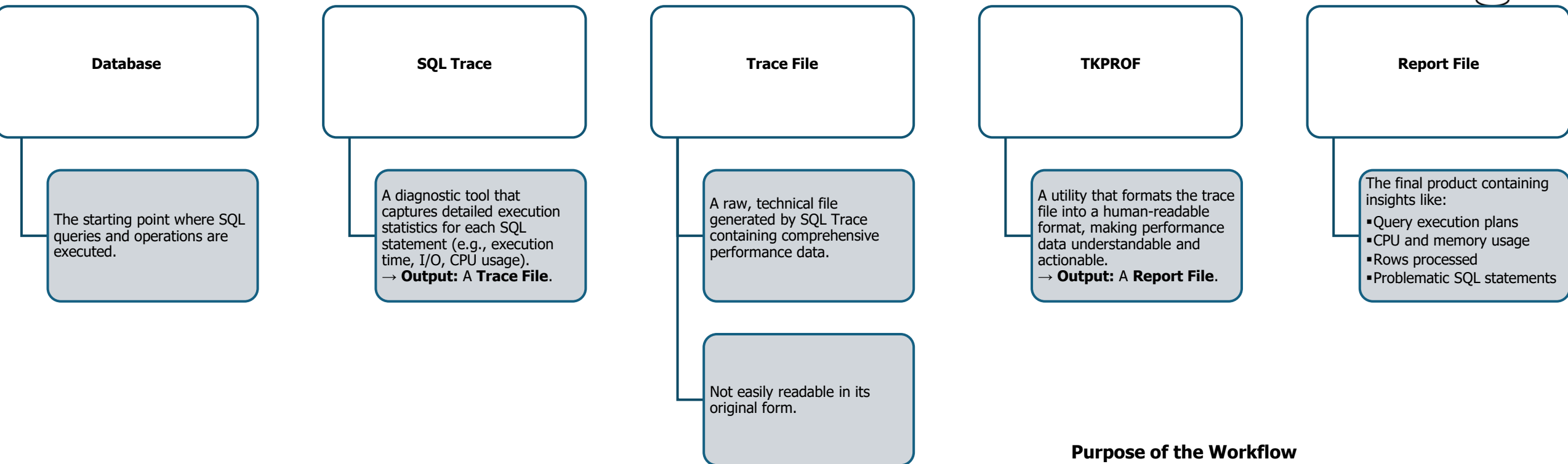
Summary: SQL Trace vs. EXPLAIN PLAN

Feature	SQL Trace + TKPROF	EXPLAIN PLAN
Measures	Actual runtime performance	Estimated execution path
Use Case	Debug slow-running queries	Preview how Oracle will execute a query
Output	Trace file + readable report	Execution plan tree
Analysis Level	Detailed runtime statistics	Optimizer decisions

SQL Performance Analysis: Tracing Facility



SQL Performance Analysis: Tracing Facility



Purpose of the Workflow

This process helps database developers and DBAs **identify and tune inefficient SQL statements**, making it essential for improving application and database performance. Let me know if you'd like this turned into a slide or formatted for documentation

Enable SQL Trace

```
Command Prompt - sqlplus s X + v

C:\Users\Eric> sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA

SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 24 06:11:03 2024
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Connected to:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL>
SQL> ALTER SESSION SET sql_trace = TRUE;

Session altered.

SQL>
SQL> EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE);

PL/SQL procedure successfully completed.

SQL>
```

*-- To connect to an Oracle database using SQL*Plus as the SYS user with SYSDBA privileges*

C:\Users\Eric>sqlplus sys as SYSDBA
or
C:\Users\Eric> sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA

--Setting SQL_TRACE = TRUE for your session:
ALTER SESSION SET sql_trace = TRUE;

--This step is already covered by the command you used
EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE);

SQL Trace and TKPROF (1/4)

```
SQL Plus
Windows PowerShell

SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 24 05:03:56 2024
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Enter user-name: system
Enter password:
Last Successful login time: Sun Nov 24 2024 04:57:14 +02:00

Connected to:
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL> EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE);

PL/SQL procedure successfully completed.

SQL> SELECT tracefile
2  FROM v$process p
3  JOIN v$session s ON p.addr = s.paddr
4  WHERE s.audsid = SYS_CONTEXT('USERENV', 'SESSIONID');

TRACEFILE
-----
C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_836
4.trc

SQL> |
```

Step 1: Enable SQL Trace for Your Session

*--First, connect to your Oracle Database using SQL*Plus or any other SQL interface.*

--To enable SQL trace for your session, execute the following command:
`EXEC DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE);`
--This will start tracing for your session, capturing wait events and bind variables.

Step 2: Find Your Trace File

--To find the location of your trace file, run the following query:

```
SELECT tracefile
FROM v$process p
JOIN v$session s ON p.addr = s.paddr
WHERE s.audsid = SYS_CONTEXT('USERENV', 'SESSIONID');
```

--This query will return the path to your trace file.

SQL Trace and TKPROF (3/4)

```
Windows PowerShell
PS C:\Users\Eric> tkprof C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc output.txt explain=system/admin

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

PS C:\Users\Eric> cat .\output.txt

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

Trace file: C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc
Sort options: default

*****
count      = number of times OCI procedure was executed
cpu        = cpu time in seconds executing
elapsed    = elapsed time in seconds executing
disk       = number of physical reads of buffers from disk
query      = number of buffers gotten for consistent read
current    = number of buffers gotten in current mode (usually for update)
rows       = number of rows processed by the fetch or execute call
*****

SQL ID: akjkx37y0xpm Plan Hash: 0

BEGIN DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE); END;

call      count          cpu    elapsed       disk    query    current    rows
-----
Parse         2         0.00         0.00         0         0         0         0
Execute      3         0.00         0.00         0         0         0         3
```

Step 3: Locate the Trace File in the File System

Navigate to the directory where the trace file is located (based on the output from the previous step). For example:

- C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace

Step 4: Use TKPROF to Format the Trace File

1. Open a command prompt or PowerShell window on your machine.
2. Navigate to the folder where the trace file is located, or directly provide the full path.
3. Run the following TKPROF command:

```
tkprof
C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc output.txt
explain=system/<password>
```

Step 5: View the Output File

After running the TKPROF command, you can view the output.txt file by using the following

```
cat .\output.txt
or
notepad .\output.txt
```

SQL Trace and TKPROF (4/4)

```
output.txt
File Edit View

TKPROF: Release 21.0.0.0.0 - Development on Sun Nov 24 05:06:24 2024

Copyright (c) 1982, 2021, Oracle and/or its affiliates. All rights reserved.

Trace file: C:\ORACLE21C_CONFIG\diag\rdbms\oracle21cdb\oracle21cdb\trace\oracle21cdb_ora_19684.trc
Sort options: default

*****
count      = number of times OCI procedure was executed
cpu         = cpu time in seconds executing
elapsed     = elapsed time in seconds executing
disk        = number of physical reads of buffers from disk
query       = number of buffers gotten for consistent read
current     = number of buffers gotten in current mode (usually for update)
rows        = number of rows processed by the fetch or execute call
*****

SQL ID: akjkyx37y0xpm Plan Hash: 0

BEGIN DBMS_SESSION.SESSION_TRACE_ENABLE(waits => TRUE, binds => TRUE); END;

call      count          cpu    elapsed       disk      query    current    rows
-----
Parse         2         0.00         0.00         0         0         0         0
Execute       3         0.00         0.00         0         0         0         3
Fetch         0         0.00         0.00         0         0         0         0
-----
total         5         0.00         0.00         0         0         0         3

Misses in library cache during parse: 0
Misses in library cache during execute: 1
Optimizer mode: ALL ROWS

Ln 1, Col 1      57,369 characters
```

Step 6: Interpret the Results

- The output.txt file will contain performance details of the SQL statements executed during your session, including execution time, CPU usage, IO statistics, and any wait events.
- The EXPLAIN section will show you how the database is executing the SQL queries, which can be helpful for performance tuning.

EXPLAIN PLAN

Command Prompt - sqlplus s



```
C:\Users\Eric>sqlplus sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA
```

```
SQL*Plus: Release 21.0.0.0.0 - Production on Sun Nov 24 06:37:50 2024  
Version 21.3.0.0.0
```

```
Copyright (c) 1982, 2021, Oracle. All rights reserved.
```

```
Connected to:  
Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production  
Version 21.3.0.0.0
```

```
SQL> DROP TABLE PLAN_TABLE;
```

```
Table dropped.
```

```
SQL> @?/rdbms/admin/utlxplan.sql
```

```
Table created.
```

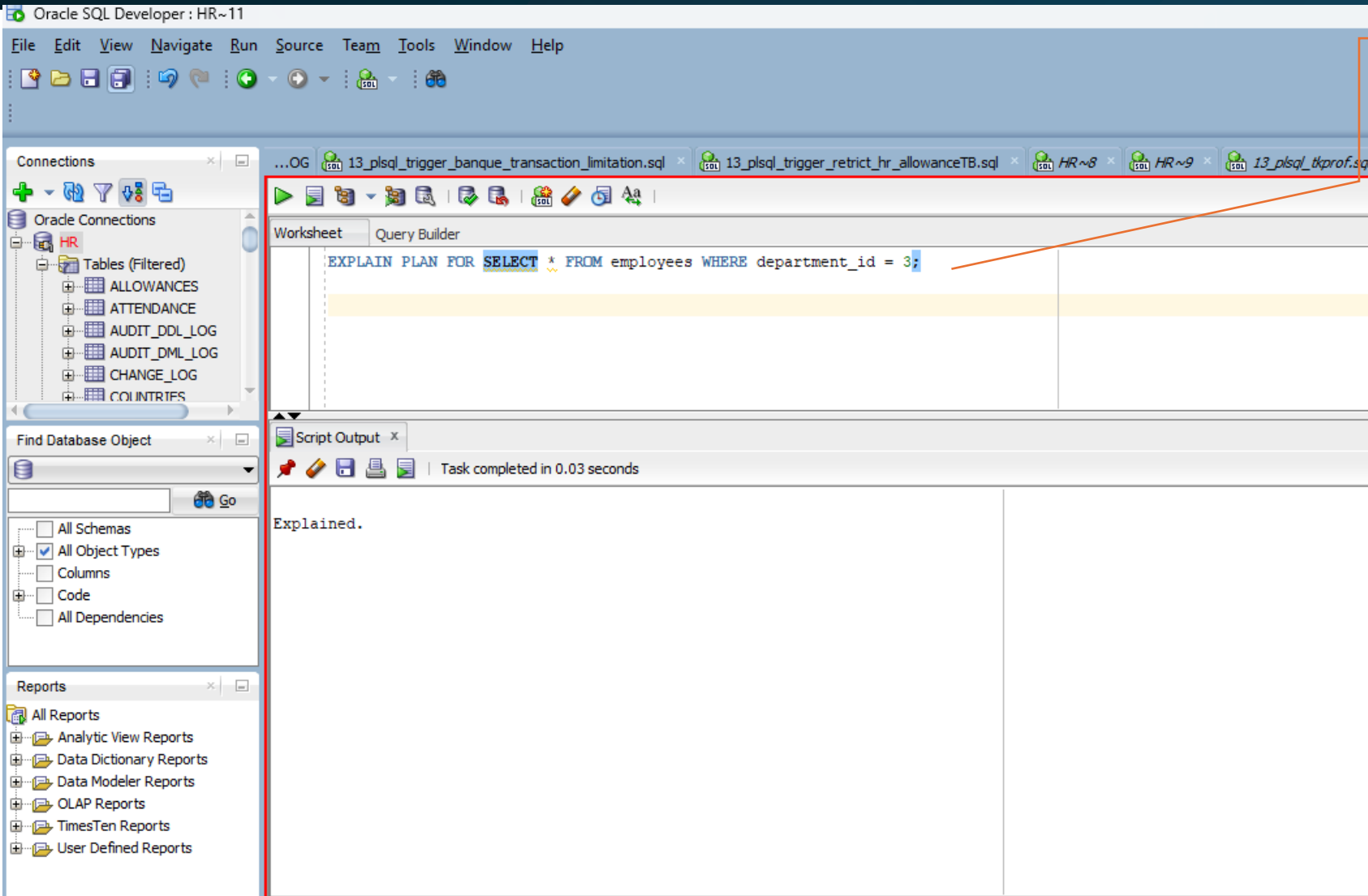
*--To connect to an Oracle database using SQL*Plus as the SYS user with SYSDBA privileges*

```
C:\Users\Eric>sqlplus sys as SYSDBA  
or  
C:\Users\Eric> sqlplus  
sys/admin@localhost:1521/ORACLE21CDB AS SYSDBA
```

--Drop existing Plan_Table
SQL> DROP TABLE PLAN_TABLE;

-- 1. Prepare the Plan Table:
Use the default PLAN_TABLE provided by Oracle, or create one using the following:
*/
@?/rdbms/admin/utlxplan.sql

Generate Execution PLAN



--2. *Generate the Execution Plan:* Use the *EXPLAIN PLAN* command:

```
*/  
EXPLAIN PLAN FOR SELECT * FROM employees  
WHERE department_id = 3;
```

View the Execution Plan

The screenshot shows the SQL*Plus interface with the following components:

- Connections:** Oracle Connections, HR, Tables (Filtered), ALLOWANCES, ATTENDANCE, AUDIT_DDL_LOG, AUDIT_DML_LOG, CHANGE_LOG, COUNTRIES.
- Find Database Object:** All Schemas, All Object Types, Columns, Code, All Dependencies.
- Reports:** All Reports, Analytic View Reports, Data Dictionary Reports, Data Modeler Reports, OLAP Reports, TimesTen Reports, User Defined Reports.
- Worksheet:** Query Builder, 13_plsql_trigger_banque_transaction_limitation.sql, 13_plsql_trigger_retrict_hr_allowanceTB.sql, HR~8, HR~.
- Script Output:** All Rows Fetched: 13 in 0.017 seconds.
- Query Result:** PLAN_TABLE_OUTPUT, 1 Plan hash value: 1445457117, 2, 3, 4 | Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |, 5, 6 | 0 | SELECT STATEMENT | | 28 | 2688 | 3 (0) | 00:00:01 |, 7 | * 1 | TABLE ACCESS FULL | EMPLOYEES | 28 | 2688 | 3 (0) | 00:00:01 |, 8, 9, 10 Predicate Information (identified by operation id):, 11, 12, 13 1 - filter("DEPARTMENT_ID"=3).



--View the Execution Plan:: Query the PLAN_TABLE to see the plan details:

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY());
```

Key Takeaways

1. The query scans all rows in the EMPLOYEES table, filtering for DEPARTMENT_ID = 3.
2. This is a relatively low-cost operation (cost = 3) since the table is small (28 rows).
3. For larger datasets, an index on DEPARTMENT_ID could improve performance.

All additional information or Partition details

The screenshot shows the Oracle SQL Developer interface. The main window displays a SQL query in the Worksheet:

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(NULL, NULL, 'ALLSTATS LAST'));
```

The Query Result tab shows the execution plan output:

```
PLAN_TABLE_OUTPUT
1 Plan hash value: 1445457117
2
3 -----
4 | Id | Operation          | Name           | E-Rows |
5 |----|-----|
6 | 0 | SELECT STATEMENT    |                |        |
7 |* 1 | TABLE ACCESS FULL | EMPLOYEES      | 28      |
8 |----|-----|
9
10 Predicate Information (identified by operation id):
11 -----
12
13 1 - filter("DEPARTMENT_ID"=3)
14
15 Note
16 -----
17 - Warning: basic plan statistics not available. These are only collected when:
18   * hint 'gather_plan_statistics' is used for the statement or
19   * parameter 'statistics_level' is set to 'ALL', at session or system level
```

The left sidebar shows the Connections pane with the HR schema selected, and the Reports pane with various report types listed.



```
/*
To display additional information like predicates
or partition details:
*/
SELECT * FROM
TABLE(DBMS_XPLAN.DISPLAY(NULL, NULL,
'ALLSTATS LAST'));
```

Key Takeaways

- The query execution plan reveals a full table scan, which is fine for small tables but should be optimized for larger datasets.
- Enabling runtime statistics allows deeper analysis and tuning opportunities.
- Consider indexing frequently filtered columns like DEPARTMENT_ID to improve performance.

PL/SQL performance tuning enhances the **efficiency** and **speed** of code execution and database interactions. It focuses on eliminating bottlenecks and optimizing resources through the following key strategies:

A large, thick green circular arrow graphic that starts on the left, curves around the top, and points down towards the bottom left, framing the summary text.

Summary of Summary: PL/SQL Performance Tuning

- **Efficient SQL:** Use **indexes**, **bind variables**, and **optimized queries** (e.g., EXISTS instead of IN).
- **Bulk Processing:** Replace row-by-row logic with **BULK COLLECT** and **FORALL** to reduce context switching.
- **Minimize Context Switching:** Reduce the number of PL/SQL-to-SQL calls by grouping operations.
- **Native Compilation:** Compile PL/SQL into machine code for faster execution speeds.
- **Profiling Tools:** Utilize **DBMS_PROFILER**, **DBMS_TRACE**, and **SQL Trace/TKPROF** to locate performance bottlenecks.
- **Result Caching:** Store frequently accessed data or query results using **PL/SQL result cache**.
- **Loop Optimization:** Use loops efficiently, avoiding unnecessary iterations and placing SQL outside loops where possible.
- **Built-in Functions:** Use Oracle's built-in functions for efficient processing and transformation.
- **Testing & Monitoring:** Continuously analyze performance using **AWR**, **ADDM**, and **V\$ views**.
- **Bottom Line:** Apply these strategies consistently to achieve significant performance gains in PL/SQL applications.

Keep Learning!